ROCK N ROLLER

Offline reference for Rock N Roller — About, Features, Stems, Keybinds, and Settings. Live site: rocknroller.nicapotato.com.

1. About
2. Features
3. Stems & Demucs
4. Keybinds
5. Settings

About

A freeware guitar-practice app that plays Rocksmith-format `.psarc` charts from your own computer — PC `*_p.psarc` and Mac `*_m.psarc`, on either operating system. Your personal library stays on disk; the app never uploads it. Built around freeware CDLC from the CustomsForge community. Grant a local folder of your own CDLC to play. Optional curated demos load from our CDN only when you open Play with `?demo=songs` (charts) or `?demo=stems` (charts + stem mixes; larger download). How to generate your own stems and mix them in-game: stems & Demucs.

Trailer

Watch trailer — youtube.com/watch?v=k--zDnyUZvs

What it does

Point it at a folder of CDLC `.psarc` files on your disk and it renders the charts on a 3D highway (with measure bar lines and capo marks) or a wrapping TabSheet, with audio, practice speed control (pitch-preserving time stretch), lead / rhythm / bass / combo arrangements, per-arrangement tuning info, playlists and profiles.

Your files stay on disk

Charts live on your machine. In the browser, the app asks for **read-only** access to a local folder via the folder picker; `.psarc` bytes are read in the page and never uploaded. Saved state (profiles, playlists, scan cache) lives in your browser's local storage for this site.

Chromium required (web)

The browser build needs a Chromium-based browser (Chrome or Edge). Folder access uses the File System Access API (`showDirectoryPicker`), which Firefox and Safari do not support. The build also runs multithreaded WebAssembly (Emscripten pthreads via `SharedArrayBuffer`), which needs a cross-origin-isolated page — something the site sets up automatically, but Chromium is still the supported target for the full library flow.

Chrome also refuses system and app-data folders (`Program Files`, `AppData`, `~/Library`) — exactly where Steam installs Rocksmith 2014. For the web player, copy `songs.psarc` and `dlc/` to an ordinary folder such as `Documents/RockNRoller` and pick that (details). Desktop builds read the Steam folder directly.

What it is not

This project is **not affiliated with Ubisoft or Rocksmith**, and it does not host, distribute, or bypass official Rocksmith DLC. It reads chart files you already have. Support official releases and the artists who make the music.

Platforms

Play in the browser at /latest (Chromium required), or grab desktop builds for macOS (Apple Silicon and Intel) and Windows from the versions page or itch.io. Desktop builds read `.psarc` folders from disk without the browser folder API.

Bugs, support, feedback

Join the Discord for bug reports and support, or to leave feedback and suggestions.

Features

How Rock N Roller solves its harder problems: local CDLC, catalog search, practice speed and section loops, Demucs stems, left-handed / inverted highway layout, a 3D chart whose camera math and note clock stay locked to the audio callback, a wrapping TabSheet that infers rhythm notation from the same beat map the highway uses for bar lines, capo indication from the arrangement manifest, lead / rhythm / bass / combo (and bonus) parts, and a Chords + Lyrics sheet for following names with the words — from one C + raylib codebase on macOS, Windows, and WebAssembly.

1. Stack & platforms
2. Library that stays on your disk (0.2.0)
3. SQLite on disk (macOS / Windows)
4. Catalog search & query language
5. Browser library access
6. Multithreaded wasm
7. Practice speed without pitch shift
8. Section loop
9. Stems & Demucs
10. Left-handed & string order
11. Lead, rhythm, bass, combo
12. Long songs without blowing RAM
13. 3D highway
    1. World space & fret math
    2. Camera, frustum, trigonometry
    3. What a frame draws
    4. Bar lines (measure starts)
    5. Notes in PSARC / SNG
    6. Load, audio, and raylib threads
    7. How the chart locks to audio
14. TabSheet
15. Capo indication
16. Chords + Lyrics
17. Keeping the UI smooth
18. What ships

Stack & platforms

The app is C17 (plus one C++17 wrapper for Signalsmith Stretch) on raylib 6.0. One CMake project builds macOS (Apple Silicon and Intel), Windows x64, and Emscripten wasm from the same commit — platform differences stay in small shims for paths, dialogs, and threading.

Heavy lifting is a short list of libraries: SQLite for state and the song catalog, an in-repo PSARC reader, vgmstream for Wwise audio (no FFmpeg), ogg/vorbis, and Signalsmith for time stretch. The UI is drawn in-app; there is no separate toolkit.

Library that stays on your disk

From **0.2.0** the catalog is **one library folder** — not a list of PSARC roots plus a separate stems picker. Point Settings at a Steam **Rocksmith 2014** install if you own the game, or at any folder with `dlc/` and `stems/`. Charts never leave your disk. Version notes: 0.2.0 changelog.

PLATFORM	EXPECTED STEAM ROCKSMITH 2014 FOLDER
macOS	`~/Library/Application Support/Steam/steamapps/common/Rocksmith2014`
Windows	`C:\Program Files (x86)\Steam\steamapps\common\Rocksmith2014` (or `%ProgramFiles%\Steam\steamapps\common\Rocksmith2014` if Steam is not in `(x86)`)
Linux	`~/.steam/steam/steamapps/common/Rocksmith2014` (or `~/.local/share/Steam/steamapps/common/Rocksmith2014`)

Settings in the app prints that expected path for the OS you are on. On desktop, **Choose library folder** opens in the `Rocksmith2014` game folder itself when it exists (not its `dlc/` subfolder), else in `steamapps/common`; nothing is bound until you confirm. **Open folder** reveals the bound root so you can drop CDLC into `dlc/`.

Web player: every path in this table sits inside a folder Chrome refuses to open (`Program Files`, `~/Library`). Copy the game files to an ordinary folder first — see Folders Chrome cannot open.

If you own Rocksmith 2014

Choose the game's install folder — the same directory Steam names `Rocksmith2014`. Official DLC already lives in that `dlc/` tree. Custom charts can sit beside it, or in a subfolder such as `dlc/cdlc/`. Stem mixes go in `stems/` next to the game.

If you do not own the game

Create any folder with the same two subdirectories. The app creates `dlc/` and `stems/` if they are missing:

```
MyLibrary/
  dlc/          # your *_p.psarc / *_m.psarc (subfolders are scanned)
  stems/        # optional Demucs mixes, one folder per chart
```

Then choose `MyLibrary` — not `dlc/` itself — and run Batch sync.

`dlc/` is recursive

Every playable `*_p.psarc` (PC) and `*_m.psarc` (Mac) under `dlc/` is catalogued, including nested folders. Either file plays on macOS or Windows. Organize however you like:

```
dlc/
  some_song_p.psarc
  cdlc/
    another_song_m.psarc
    artist/
      third_song_p.psarc
```

Hidden folders (names starting with `.`) are skipped. Root game archives that sit next to `dlc/` are not catalogued — only files under `dlc/` count. After you add or move charts, run **Batch sync** again.

`stems/` — six MP3s per chart

Stem mixes live under the same library folder, not a second picker. Rock N Roller looks up the chart filename without `.psarc`, then requires **all six** MP3s at `{library}/stems/{psarc_basename}/` (drums, bass, other, vocals, guitar, piano). Example: `dlc/cdlc/Eagles_Hotel-California_v1_1_p.psarc` matches `stems/Eagles_Hotel-California_v1_1_p/`.

Opening every `.psarc` on every launch would be slow. Instead, a local SQLite DB caches each song's metadata (title, artist, tunings, arrangements, length) keyed by file size and mtime. Startup loads the catalog from the DB; a background rescan only re-parses files that changed. Profiles, playlists, play history, and settings live in the same database. Charts are Rocksmith's binary SNG — parsed once when you pick a song, not from XML at runtime.

SQLite on disk (macOS / Windows)

On desktop, the database is a normal file on your machine — not in the cloud, not inside the app bundle. The state directory is created on first launch; the DB file is `rocknroller.db` (plus SQLite WAL sidecars while the app is open).

PLATFORM	PATH
macOS / Linux	`~/.nicapotato-app-saved-state/rocknroller/rocknroller.db`
Windows	`%LOCALAPPDATA%\nicapotato\rocknroller\rocknroller.db`
Web	Same logical path inside an IDBFS mount, persisted to the browser's IndexedDB for this site (see browser library access)

That file holds the song scan cache, profiles, playlists, play history, the library folder path, autoplay mode, chart layout, and per-profile cell settings (including left-handed / invert). Your PSARC and stem audio stay wherever you put them — only metadata and prefs are in the DB. Reset in Settings wipes this state, not your media files.

Catalog search & query language

A large CDLC library is useless if you can only scroll it. The catalog search bar accepts plain text *or* a small query language that filters on the metadata already in SQLite — no reopening PSARCs.

Bare words are a global substring match (case-insensitive) across title, artist, album, year, and tuning. Column filters use `field="value"` and can be mixed with free text:

CLAUSE	FILTERS ON
`title="…"`	Song title
`artist="…"`	Artist
`album="…"`	Album
`year="…"`	Year
`tuning="…"`	Tuning (display label or raw offsets), e.g. `tuning="Drop D"`
`instruments="…"`	Arrangements present: `v` vocals, `l` lead, `r` rhythm, `b` bass, `c` combo (lead + rhythm on one chart). Bonus parts: `+l`, `+r`, `+b`, `+c` (any extra of that type), `+l2` (at least two bonus leads), `+` (any bonus). Optional per-arrangement tuning: `l-E Standard`, `c-Drop D`, `+r-Drop D`
`official="true|false"`	Official Rocksmith DLC vs CDLC
`stems="true|false"`	Songs with all six external stem MP3s indexed
`psarc="m|p"`	Mac (`*_m.psarc`) or PC (`*_p.psarc`). `psarc="m"` is Mac only. A file is only one platform, so `psarc="p|m"` and `psarc="p,m"` both show either.

Inside the quotes, **comma means AND** and `|` **means OR**. Examples:

- `stems="true" tuning="E Standard | Drop D"` — songs with stems whose tuning matches either string
- `artist="Rush" instruments="l, b"` — Rush charts that have both lead and bass
- `instruments="l-E Standard|b-Drop D"` — lead in E Standard *or* bass in Drop D
- `official="false" stairway` — CDLC only, plus free-text “stairway”
- `psarc="m"` — Mac archives only. `psarc="p,m"` or `psarc="p|m"` — PC and Mac

Clicking an album or artist card fills the bar with matching `artist="…"` / `album="…"` clauses. The same query powers the play queue when autoplay advances to the next song.

Browser library access

The browser has no filesystem, but the app needs a folder of your CDLC. The solution:

- Chromium's File System Access API grants **read-only** access to a local folder; the handle is remembered in IndexedDB so you don't re-pick every visit.
- The folder is mirrored into wasm MEMFS as **zero-byte skeleton files** with real names, sizes, and mtimes. Desktop scan/cache logic runs unchanged — and because the cache keys on size + mtime, cataloging never needs the file bytes.
- Opening a song stages the real PSARC on demand: a worker parks while the main thread copies bytes into a bounded LRU cache (512 MiB), then wakes. Large libraries stream through a fixed footprint.

Bytes go disk → page memory only. Nothing is uploaded. This (and folder picking) is why the web build needs Chrome or Edge.

Folders Chrome cannot open

The File System Access API refuses system and app-data folders, and Steam installs Rocksmith 2014 inside one of them. The web build cannot read your game folder in place:

- **Windows:** `C:\Windows`, `C:\Program Files`, `C:\Program Files (x86)` (default Steam location) and `AppData`
- **macOS:** `~/Library` (Steam uses `~/Library/Application Support/Steam`) and the system `/System` and `/Library` folders
- **Linux:** `/dev`, `/sys` and `/proc`

Copy `songs.psarc` and the `dlc/` folder (plus `stems/` if you have them) from `…/steamapps/common/Rocksmith2014` to an ordinary folder such as `Documents/RockNRoller`, then choose that folder in the player. If you pick a blocked folder, the player says so instead of silently failing. Desktop builds have no such limit and open the Steam folder directly.

Multithreaded wasm

The web build is the same C, compiled with Emscripten as multithreaded wasm: pthreads → Web Workers sharing memory via `SharedArrayBuffer`. That requires a **cross-origin-isolated** page (COOP/COEP headers). GitHub Pages can't send those, so a same-origin `coi-serviceworker` injects them on player URLs only (`/latest/`, `/0.1.35/`, … — one reload on first Play visit). Landing and docs stay un-isolated so embedded YouTube can load. The ~4 MB engine then streams from CDN; the site itself stays a thin shell.

Practice speed without pitch shift

Pitch is frequency. An **octave** up means the waveform repeats twice as often: 220 Hz (A3) → 440 Hz (A4) → 880 Hz (A5). Twelve equal-tempered semitones make that doubling:

```
f′ = f · 2^n   (n = octaves)
one semitone = 2^(1/12) ≈ 1.0595×
```

Playing a buffer slower without processing stretches each period `T`. Frequency is `1/T`, so half speed makes every note an octave down and the guitar sounds like a bass. Practice mode needs **0.3×–3.0×** with that frequency held still — the lick lasts longer (or shorter), but A stays A.

Inside the miniaudio callback the app does not resample. It asks Signalsmith Stretch (cheaper preset, `splitComputation` on) to emit a device-rate period from a different number of source frames:

```
in_frames = ceil(device_frames · rate)
stretch(in, in_frames) → out[device_frames] at the original pitch
```

At 0.5× it pulls half as much song audio as it plays to the DAC: the lick is twice as long. At 2× it pulls twice as much and compresses it into one period. At **1.0× the stretcher is bypassed** (copy samples straight through) so there is no extra latency or FFT cost when you are not practicing.

Stretch keeps pitch by working in the frequency domain, not by scaling the waveform. Overlapping windows (~100 ms block, ~40 ms hop at 48 kHz) go through an STFT. Each bin is a sinusoid with a magnitude and a phase. Time-stretch walks the *analysis* windows through the input at `rate` while the *synthesis* hop stays at the device interval:

```
input_offset = output_index · (in_frames / out_frames)
```

A classic phase vocoder then advances each bin's phase so its instantaneous frequency (how fast phase rotates) stays the same as in the source. The note's harmonics keep their Hz; only how long they ring changes. Signalsmith adds a second prediction that locks phase *across neighboring bins* (vertical locking), which keeps transients and stacked guitar partials from smearing the way a naive vocoder does. Rock N Roller never transposes: the engine's frequency multiplier stays 1, so it skips the spectral-peak remap used for pitch shift. Above a 2× stretch ratio it starts randomising some phases; 3.0× is inside that "less clean" region on purpose so fast-forward still works.

`splitComputation` spreads one STFT's analyse / predict / synthesise steps across the callback period so a 1024-frame quantum does not spike the audio thread. The overlap-add still needs a look-ahead buffer; that **output latency** (synthesis delay plus one hop when split) is subtracted when publishing the heard clock, which is why the chart clock can lock gems to what you

actually hear. Speed and seek reset the stretcher atomically inside the callback so a rate change cannot mix two phase histories.

Section loop

Rocksmith charts tag named **sections** (intro, verse, solo, …). Press **P** or the footer **Loop** button to loop the section under the playhead: the app seeks to that section's start and repeats until you turn loop off. While loop is on, **Enter** or **O** jumps back to the loop start and unpauses — useful after you scrub away mid-practice.

Loop edges show on the seek bar and can be dragged to tighten the span (as long as the chart has sections). Combined with pitch-preserving speed and stem mutes, this is the usual "drill this four bars" workflow. See keybinds.

Stems & Demucs MLX

Stem mode mixes six isolated tracks (drums, bass, other, vocals, guitar, piano) with per-stem faders. Separation runs offline in the companion Demucs MLX app (macOS Apple Silicon via MLX, or Windows x64 via NVIDIA CUDA), then Rock N Roller matches `{library}/stems/{psarc_basename}/` and streams the six MP3s. Full workflow, Mac vs Windows size/speed, folder layout, mixer, and `?demo=stems`: stems page.

Left-handed & string order

Two independent highway layout flags, saved per profile and per cell size class (also bulk-set from catalog Settings):

SETTING	DEFAULT	WHAT IT DOES
String order (invert)	Red / low-E at **top**	Rocksmith-style default. **Inverted** puts red at the **bottom** of the lane — useful if you prefer that visual mapping.
Left-handed	Right-handed	Mirrors fret positions about the neck midpoint (Rocksmith-style lefty), so open/low frets sit on the opposite side of the highway.

Toggle them in Cell Settings while playing, or use Settings → "Set all to Left-handed / Right-handed" and "Set all to Default / Inverted" for every size class at once. Details: settings.

Lead, rhythm, bass, combo

A Rocksmith archive can ship more than one guitar-family chart. Rock N Roller reads the **manifest next to each SNG** (`ArrangementName`, `bonusArr`, `represent`) and maps each part to a fixed slot. Filenames are only a fallback when there is no manifest.

SLOT	LETTER	WHAT IT IS
Vocals	`V`	Lyric syllables (karaoke / Chords + Lyrics)
Lead	`L`	Primary lead arrangement
Rhythm	`R`	Primary rhythm arrangement
Bass	`B`	Primary bass (4 strings)
Combo	`C`	Lead and rhythm authored on *one* chart — a single arrangement between the usual lead and rhythm parts, not a merge of the two SNGs at runtime

Combo is its own `ArrangementName` (`Combo` / `*_combo*.sng`). It sits in the instrument dropdown after Lead / Rhythm / Bass (those three are always listed, and show **N/A** when the song lacks them). Combo and bonus rows appear only when the archive has them. Bonus / alternate parts of each type fill extra slots: `+L`, `+L2`, `+R`, `+B`, `+C`, … The catalog INSTR column prints `V L R B C`, then dimmed bonus tags; a narrow column collapses extras into one `+`.

Saved layouts store integer source ids (1–4 unchanged, 5 = combo, 10+ = bonus), so older saves load as they did. The same letters drive `instruments="…"` — see catalog search. Any guitar-family slot (lead, rhythm, bass, combo, or a bonus) can flip between the 3D highway and TabSheet.

Long songs without blowing RAM

A long chart can be huge if kept fully renderer-ready. Songs over **10 minutes** keep the parsed chart in a compact cache and only expose a **3-minute window** to the highway. As you near the edge (with preload that scales with practice speed), the window slides and refills. Shorter songs drop the cache after the first fill.

Audio streams similarly: OGG from disk, stems as MP3 with seek points, and long WEM-only tracks decode in the background while the chart starts. Memory stays flat for a three-minute song or a forty-minute one.

3D highway

Each instrument pane is its own scissored 3D viewport (raylib `rlgl`). The neck is a real 3D scene — not a 2D strip with fake perspective. Gems live on a guitar-scaled fretboard, time maps to depth, and an off-axis camera looks down the runway without keystoning the fret wires. Zoom, runway grade, and lookahead are per-cell settings. Left-handed and invert flags remap X/Y before draw (see left-handed & string order). Target is 60 FPS with four panes up.

World space & fret math

The highway uses a right-handed world with a 24-fret neck:

AXIS	WHAT IT IS	INCREASING TOWARD
`X`	Along the fretboard	Higher frets (open/nut at 0; lefty mirrors this)
`Y`	Across the strings	Default: high-E at the bottom of the pane, low-E (red) at the top. Invert swaps that.
`Z`	Song time, as depth	Hit line at `z = 0`. Upcoming notes sit at *negative* Z and travel toward the camera.

Fret positions are the equal-tempered guitar formula (the same math as a real neck): each twelve-fret octave halves the remaining string length. With scale length `L = 2.25` world units:

$$x(f) = L \cdot (1 - 2^{-f/12}) \quad \text{for fret } f = 0\ldots12$$

That is `L - L / 2^(f/12)`. Consecutive fret gaps shrink toward the body the way a physical guitar does. Past fret 12 the remaining gaps would be too tight on screen, so the renderer **stretches** everything beyond the 12th fret by 1.65× around that anchor — high-fret phrases stay readable without breaking the low-fret spacing players already know.

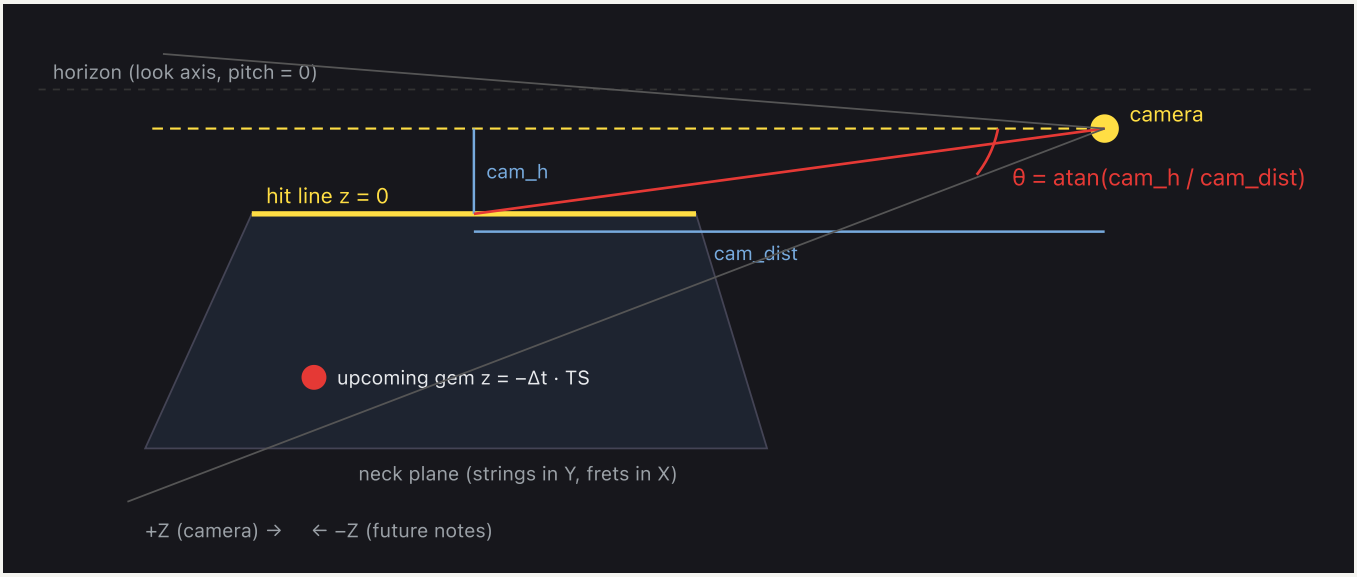
A gem's X is the midpoint of its fret cell; open notes (fret 0) are a wide slab centered on the chart's current hand-position *anchor* (the SNG "where is the left hand" window), not on the nut. String Y is evenly spaced. Time becomes depth with a constant seconds-to-world scale `TS ≈ 1.5` (200 × the neck's unit `K`):

```
Δt = note.time − song_timer
z = −Δt · TS
```

A note 2 seconds away sits at `z ≈ −3`. When `Δt ≤ 0` the gem is pinned at the strike line (`z = 0`) for the sustain, then fades. The visible window is about **0.5 s behind** the playhead to **lookahead seconds ahead** (default 3 s, slider 1.5–6). Only that slice is batched.

Camera, frustum, trigonometry

The camera never pitches. Position and look-at share the same Y, so the view direction is straight down `-Z` — horizontal. Fret wires stay parallel to the window's top edge; there is no keystone. The "looking down a rising highway" feel comes from two things: the camera sits *above* the neck, and the projection is an **asymmetric (shift) frustum** that slides the image down so the neck sits low in the pane with a horizon near the top.



Side view. The camera looks horizontally; the neck sits below the look axis. Depression to the strike line is θ = atan(cam_h / cam_dist). Raising the camera (runway grade) increases θ and the highway looks steeper.

Runway grade is that height multiplier. Default 1.0 matches the original elevation; the slider (0.35–2.2) scales `cam_h` only. Higher grade → larger θ → the runway converges toward the horizon faster. The camera still does not pitch.

Zoom is mostly a dolly: `cam_dist` is divided by the zoom factor (0.5×–2.5×). Closer camera, larger on-screen neck. A small FOV tweak then fits the residual.

Perspective uses the usual half-angle tangent. With vertical field of view `fovy` and near-plane distance `n`:

```
top = n · tan(fovy / 2)
right = top · (viewport_width / viewport_height)
```

A symmetric frustum would be `[-right, right] × [-top, top]`. Instead the renderer shifts the vertical window by a fraction `s` of `top` (a shift-lens / off-axis projection):

```
cy = top · s
frustum Y = [cy − top, cy + top]
```

`s` is not a user slider. After FOV is fitted, it is solved in one closed form so the near-bottom of the neck (fret-number row) pins to a fixed screen Y — about the bottom of the pane — regardless of zoom or grade. A uniform vertical shift moves every projected Y by the same amount, so it cancels out of the FOV height ratio and can be applied last. Pitching the camera would have keystoned the frets; shifting the frustum does not.

Half the world-space width visible at the camera's distance (the quantity the wide-pane pan uses):

```
visible_half = tan(fovy / 2) · cam_dist · aspect
```

Two camera-follow modes sit on top of that fit. **Center** (tall / quarter cells) keeps the active lane in the middle. **Wide** (full-width cells) holds a deadband: the camera only pans when the required fret span approaches a frustum edge, then glides the minimum amount. Lane extents themselves are a box-filtered average of SNG hand anchors over a short time window, so the pan is piecewise-linear in song time — no hitch when the hand jumps a few frets.

Far down the runway, perspective would shrink a constant-size gem below a pixel. World size needed for roughly constant screen size grows linearly with depth, so open bars and thin wires use `1 + |z| / cam_dist` as a keep-alive scale (capped so the spawn end does not balloon). Approaching gems also rotate: they start ~90° edge-on at lookahead and flatten to the neck as `(Δt / ahead) · π/2` → 0.

What a frame draws

`highway_3d_draw(instrument, song_timer, viewport, …)` runs on the raylib thread once per pane, per frame:

1. Scissor to the cell; rebuild a per-frame cache (lane bounds, note batch) keyed on `song_timer`.
2. Binary-search the sorted note array for `[song_timer − 0.5s, song_timer + ahead]`, including sustains that started earlier and still overlap (cap 512 visible notes).
3. Fit / reuse the camera; begin a custom 3D mode with the shifted frustum (not stock `BeginMode3D`, which cannot scissor per pane or shift the lens).
4. Draw in a fixed depth order: lane surface → bar lines → hit bar → fret wires → capo shade / capo bar → glass chord frames → sustain ribbons (depth-write off) → string wires → gem halos / ghost repeats → solid gem bodies → open slabs (depth test off so far bars do not flicker against the faded lane).
5. Restore the 2D ortho matrix and overlay SDF fret numbers, chord names, and the chord-diagram HUD in screen space (world → clip → NDC → pane pixels, using the same shifted projection).

Gems are halo / body / edge, Rocksmith-style. Sustains are ribbons that can follow a bend envelope. Repeat-chord "ghost" gems travel inside the next identical frame at reduced alpha so

they never depth-block the landing chord. Four panes share four small camera contexts so layout A does not steal layout B's pan state.

60 FPS is a 16.7 ms budget for the whole window — every highway, the lyrics pane, headers, and the footer. The bottleneck is almost never "too many triangles." It is **GPU submissions**: each time GL state changes (texture, shader, blend, depth-write, scissor, viewport, quad vs triangle), raylib's `rlgl` must flush the vertex batch and start a new draw call. A naive loop that fully drew each gem (halo + body + edge + icon + label) would flush hundreds of times per pane.

The note **batch** exists to stop that. Visible notes are collected once into a `Hw3dNoteBatch`, then the same array is walked in several cheap passes. Within a pass, gems share one `rlBegin(RL_QUADS)` (or triangles) so four edge bars and many gem faces land in a single submission instead of a `DrawCube` per note. Technique icons go into a spritesheet atlas and are drawn after 3D with one texture. Lane/camera caches skip work when `song_timer` and the viewport have not changed in a way that matters. Depth-write still has to toggle between opaque gems and glass/ribbons — those flushes are deliberate, not accidental — but they happen once per *layer*, not once per gem.

That is why the full kitchen sink is hard. A **four-quarter** layout is four independent 3D viewports: four scissors, four projections, four depth clears, four note batches, four 3D→2D restores, four SDF fret-label passes and chord HUDs. Cost is roughly linear in pane count, and a dense chart at long lookahead can fill all 512 slots in each cell. Put **lyrics** (or **Chords + Lyrics**) in the remaining slot — the "3 highways + lyrics" template, or **3D + Lyrics** as an inline strip on a wide cell — and you add a different tax on the same 16.7 ms: the SDF font shader, a `DrawTextEx` per syllable, karaoke bloom (the active word is drawn five times, center plus four offsets), and chord names drawn twice for weight. Every shader / 2D-vs-3D flip breaks the batch the highways just built. Four quarter highways alone already multiply the 3D path; four cells of highways *plus* a chords-and-lyrics pane is the worst of both worlds, which is why the 60 FPS target is aimed at that layout, not at a single full-window neck.

Bar lines (measure starts)

Each **measure** in the chart is one horizontal strip across the 3D lane — a cyan line that travels toward the hit bar with the notes. It is not a beat tick. A 4/4 bar therefore shows one line every four beats, not one per beat.

The source is the SNG **ebeat** table (16 bytes per row: time, measure, beat, phrase iteration, mask). Every beat stores a 0-based bar number in `measure`, so that field repeats for the whole bar and cannot mark a downbeat on its own. Only `mask & 1` (`BEAT_FIRST_OF_MEASURE`) does. On copy into the GUI, those downbeats keep a 1-based bar number; other beats get `measure = -1`. Charts with no mask bits fall back to "the bar number changed." The same downbeats become TabSheet measures — see TabSheet.

At draw time the highway walks beats in `[song_timer − behind, song_timer + ahead]`, skips any beat that is not a downbeat, and also skips a downbeat sitting on the strike line (the hit bar already marks "now"). Each remaining downbeat becomes `z = −Δt · time_scale`, a pair of lane-width quads (soft halo + brighter core) in the same cyan as the highway rails. Thickness and alpha grow with distance so a far bar stays readable instead of fading to a hairline. Cell Settings can hide them (**Bar lines (measure starts)**, on by default).

Notes in PSARC / SNG

A `*_p.psarc` (PC) or `*_m.psarc` (Mac) is a PlayStation-style archive: magic `PSAR`, zlib block compression, a table of contents, and a names block. Official DLC sets archive flag 4 and AES-encrypts the TOC with the same key on both platforms; CDLC usually does not. The reader decrypts when needed, inflates each entry on demand, and never needs the XML toolkit files at runtime. Either file plays on macOS or Windows.

Chart data lives in `songs/bin/…` as binary **SNG** (not XML):

- `*lead*.sng`, `*rhythm*.sng`, `*bass*.sng`, and when present `*combo*.sng` (lead + rhythm on one chart — see arrangements)
- `*vocals*.sng` for lyrics
- Bonus / alternate parts of each type (`*lead2*.sng`, `+R`, `+C`, …)

An SNG file on disk is still encrypted. Header magic `0x4A`, a 16-byte AES-CTR IV, then ciphertext with a platform key: PC for `*_p.psarc` (charts under `songs/bin/generic`), Mac for `*_m.psarc` (charts under `songs/bin/macos`). Audio follows the same split (`audio/windows` vs `audio/mac`). If the same song exists as both files, the catalog lists both and tags only the Mac row with **M**. After decrypt, a u32 uncompressed size and a zlib stream unpack to the real chart blob: BPM, then one *arrangement* per dynamic-difficulty tier, each with its own notes and hand anchors, plus song sections (intro / verse / solo), chord templates, and (for vocals) timed lyric syllables.

Playback uses the arrangement whose difficulty equals the SNG metadata max — the fully authored chart, not a lower DD tier. Each note row is a packed binary record. The fields the highway actually stores:

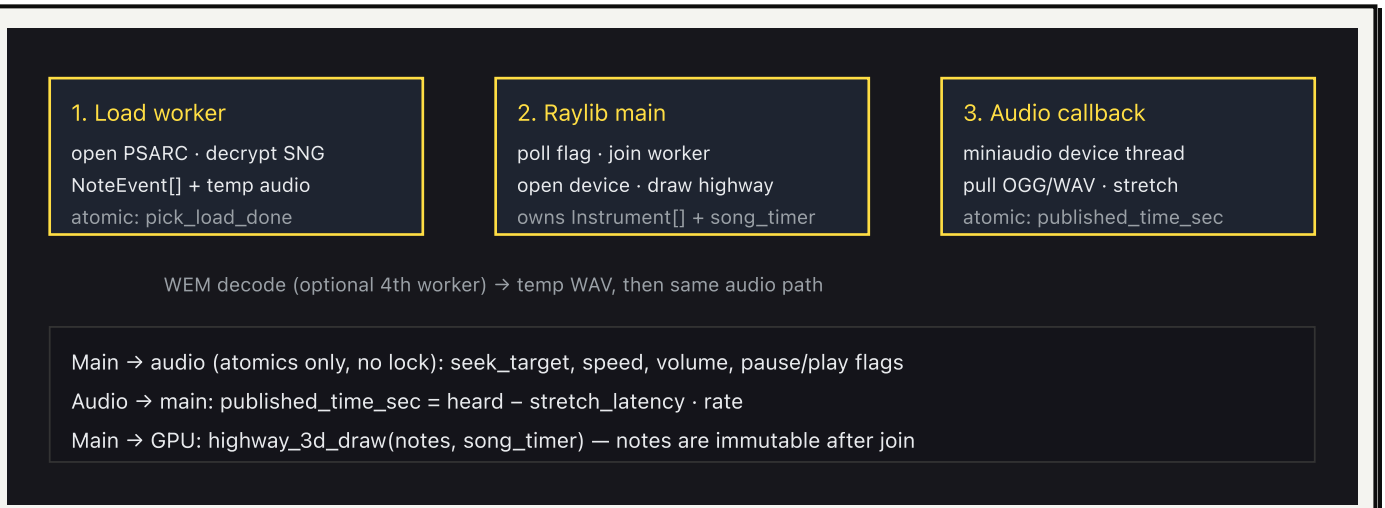
FIELD	ROLE ON THE HIGHWAY
`time` (seconds)	When the gem should hit; becomes Z via `−Δt · TS`
`string` (0 = low E)	Lane Y (after invert)
`fret`	Lane X (0 = open bar)
`sustain`	Ribbon length; gem parks at the hit line while held
`note_mask`	Techniques: hopo, bend, slide, mute, harmonic, tap, unpitched slide, …
`anchor_fret / width`	Left-hand window; camera framing and open-string bar width

FIELD	ROLE ON THE HIGHWAY
`chord_id` + templates	Named chord frames / HUD; chords expand to per-string gems
bend times / steps	Ribbon shape while the note is held
`slide_to` / unpitched	Trail destination fret

Those rows are copied into a flat `NoteEvent[]`, sorted by time then string. Chord SNG rows explode into one gem per sounding string (a “spoke”). Songs longer than 10 minutes keep the full parsed SNG in a compact cache and only expose a 3-minute window to the highway — see long songs. Audio is a separate extract: first `*.ogg` if present, else the largest `audio/*.wem`, decoded with vgmstream to a temp WAV.

Load, audio, and raylib threads

Three threads touch a song. They never share OpenGL, and the audio callback never waits on the renderer. That split is why Esc back to the catalog can keep the mix playing while you browse Library / Albums / Artists / Settings.



The PSARC worker never talks to the audio device. Main joins the parse, then starts playback. From then on the callback and the renderer share only atomics.

Picking a song spawns `pick_load_thread`. That worker is the only place that opens the PSARC, decrypts SNGs, and fills `Instrument.notes`. The frame loop keeps rendering the catalog (or a loading overlay) until `pick_load_done` flips; then main *joins* the worker (so the note arrays are visible) and `music_reload` opens miniaudio on the *main* thread. Device creation is not done on the loader — audio APIs want the thread that owns the player.

After that, a dedicated **device callback** (miniaudio, not raylib’s audio) runs at the hardware period (~1024 frames). It pulls source samples, optionally time-stretches them, writes the output buffer, and publishes a clock. Seek, speed, and volume from the UI are atomic stores plus a pending flag; the callback consumes them at the start of a period. No mutex, so a hitch in draw

cannot underrun the DAC. Long WEM tracks decode on a fourth worker; the chart can start before the WAV exists, then the callback is swapped in when decode finishes (see keeping the UI smooth).

On web this is the same C: pthreads become Workers on `SharedArrayBuffer` (see multithreaded wasm). The "raylib thread" is still the browser's main thread that owns WebGL.

How the chart locks to audio

The highway does not read the DAC clock every frame. Audio periods are coarser than 60 Hz and stretch adds latency, so a raw sample-counter would make gems stutter. Instead main keeps a **chart clock** that is *anchored* to audio and *extrapolated* with wall time between samples.

1. The callback tracks `heard` (how far the decoder has read into the file). When stretch is active it subtracts `(latency_samples · rate) / sample_rate` so the published time is "what you are hearing now," not "what was just pulled into the stretcher." At 1.0× stretch is bypassed and latency is zero.
2. That value is stored in `published_time_sec` (relaxed atomic). Main's `play_audio_time_played()` is just an atomic load.
3. On play / resume / seek, main snapshots `(wall_now, audio_t)` as the anchor and sets `song_timer = audio_t`.
4. Each playing frame: `song_timer = audio_t + speed · (GetTime() − wall_anchor)`. Gems move smoothly even if the callback has not run this frame.
5. About every **75 ms**, or sooner if extrapolated time drifts more than **120 ms** from the published clock (or the anchor is older than 100 ms), main re-reads the atomic and re-anchors. That corrects stretch, OS scheduling, and speed changes without a visible snap.

```
song_timer(t) = audio_anchor + speed · (wall(t) − wall_anchor)
resample when |song_timer − published_time| > 0.12 s
```

Seek writes a pending target into the callback; main holds `song_timer` on the requested time for a short grace so the highway does not jump to stale audio then back. If the published clock stops advancing while the device still claims to be playing (tab freeze, AudioContext stall), the chart clock **holds** at the last audio time instead of running ahead on wall time — that was the old "highway sawtooth" where gems raced then snapped back every resample. Speed changes reset the stretcher and re-anchor in the same frame so notes stay under the hit line at 0.3× and at 3.0×.

The draw call is then just geometry: `highway_3d_draw(inst, song_timer, pane)`. Same `song_timer` drives lyrics, the seek bar, section loop, TabSheet's playhead, and every instrument cell, so four highways cannot drift apart.

TabSheet

TabSheet is a wrapping, EXPLORE-style guitar tab for one instrument cell. The 3D highway is live sight-reading; TabSheet is the page you pause on to see a whole section at once. Click the **TAB** toggle next to the cell's instrument button (shown as **T** in compact headers). It flips that cell between 3D and TabSheet and stays filled while tab is on. Picking another instrument keeps the current view, so **2 Tall** can show the highway on top and the tab below — including on a combo part.

Saved as `source:viz` with `viz = 4`. Tab zoom is its own cell setting (50–250%), stored as `tab_zoom`. The Cell Settings **Zoom** slider edits tab scale while the cell is on TAB, and highway zoom while it is on the highway — they do not share a value. Details: settings.

The page is a dark staff (6 strings, 4 on bass) with string-colored fret numbers, the same palette as the highway. Measures pack left to right, then wrap. One bar number sits in the left margin of each row. The current bar is highlighted; a playhead walks through it. Click a bar to seek to its start (header, footer, and dropdown clicks never fall through). Wheel or shift-drag scrolls; follow resumes after a few seconds.

Techniques sit on the note: slides, hammers / pulls (**H** / **P** in the lane above the staff), bend `^n` and vibrato `~` as a small raised suffix after the digit, mute `X`, harmonics, and vertical chord names. A sustain or slide tail stops short of the next note on that string. Busy bars widen until neighbouring digits no longer collide (up to 3× the base width, or one full row). Time stays linear inside each bar, so the playhead and the rhythm stems share the same `x`.

How the musical notation is determined

SNG charts give *note times* and a *beat grid*, not written note values. There is no Guitar Pro duration field to draw. `chart_rhythm_build` (a port of quickRS `rhythm.rs`) infers the rhythm row under the staff:

1. **Snap.** Each distinct onset (the notes of a chord share one) snaps to the nearest sixteenth of its beat, or to an eighth-triplet slot when that fits the beat clearly better.
2. **Length.** A note lasts until the next onset. A short note before a long gap becomes the note (sustain decides how long it rings, at least to the end of its beat) plus rests.
3. **Notation.** Each note or rest splits at bar lines (and at the beat when it starts off the beat), then is written greedily as whole / half / quarter / eighth / sixteenth / 32nd values with dots. Split notes are tied. Neighbouring eighths-or-shorter in the same beat are beamed; a rest breaks the beam. A `3` sits under triplet beats.

The bar grid itself is the same SNG downbeats that draw highway bar lines: `BEAT_FIRST_OF_MEASURE` starts a measure, so a 4/4 bar holds four beats. Fret numbers and stems are both placed at the **snapped** time, so a note played just before a downbeat is written on that

downbeat, in the same bar as its rhythm symbol. When a note sounds, its digit and its rhythm symbol flash brighter (colour only — glyphs never grow) and fade across the written length.

Capo indication

Capo fret comes from the arrangement manifest (`CapoFret`), per part — a bonus chart can differ from the primary. `0` means none. Sync stores the song-level value on the catalog row so the library can show it without reopening the PSARC.

In the catalog a capo song gets a purple **C*n*** pill (e.g. `C4`) next to the official / stems / Mac tags. On the 3D highway the same purple marks the neck: the stretch from the nut to the capo is shaded, a bar sits 72% of the way across that fret, and a **CAPO n** label rides the bar in screen space. Open-string pulses stop at the capo fret instead of the nut, so a ringing open string does not cross the clamp.

TabSheet still prints the chart's authored fret numbers (Rocksmith already writes them relative to the capo). The catalog pill and the highway bar are how you see that a clamp is on.

Chords + Lyrics

A lyrics pane can show the chart's named chords sitting on the words — Ultimate Guitar-style changes, timed to the same vocal line cache the karaoke panel already uses. This is a **Lyrics cell** viz, not the 3D highway: glass chord frames on the neck stay their usual pale rim; only the sheet's alphanumeric names are color-coded.

Pick a layout that has a lyrics slot (e.g. 3 highways + lyrics, or Full with the cell source set to Lyrics). In that cell's header, set viz to **Chords + Lyrics** (the other lyrics option is the plain karaoke panel). Lyric size and lyric-line count still come from Cell Settings.

Names are the chart's chord-template labels (e.g. `A7sus4`), emitted on each change and restated after a stretch of silence so a verse does not stay unlabeled after a break. The chord track prefers the **rhythm** arrangement, then **lead**. If the song has no named chords (or no rhythm / lead notes), the pane falls back to the regular lyrics panel.

Each lyric row gets a chord band above the words. A change anchors above the first word that starts at or after that time; a chord in the second before a line's first word is a pickup at the line start. Colors are kept apart from karaoke (yellow past / red sweep / white upcoming):

CHORD NAME	COLOR	MEANING
Upcoming	Sky blue	Not yet sounding

CHORD NAME	COLOR	MEANING
Active	Bright green	The last named chord whose time has passed
Past	Dim slate	Already superseded

Instrument cells can still use **3D + Lyrics** for an inline strip on the highway; that strip is lyrics only. Layout and viz details: settings.

Keeping the UI smooth

The main thread renders; workers handle library sync, song load, WEM decode, stem mixer open, and album art. They finish via atomic flags the frame loop polls — no blocking joins on the hot path. Speed, seek, and volume cross into the audio callback with atomics only, so the UI can't stall playback. The same model runs on web as the pthread pool above. How those three threads hand a song to the 3D highway (PSARC parse → device callback → chart clock) is under 3D highway.

The callback is also why you can leave a chart and keep hearing it. **Esc** (or the catalog control) only drops the 3D chrome — the play session stays live, the catalog footer still seeks / pauses / changes speed, and you can switch **Library**, **Albums**, **Artists**, and **Settings** without pausing the DAC. Search, album cards, and settings are raylib-thread work; they never wait on audio.

Picking a different song spawns `pick_load_thread` for the PSARC / SNG parse. Until that worker flips `pick_load_done` and main calls `music_reload`, the previous mix keeps playing. Long WEM tracks then decode on a fourth worker (**DECODING AUDIO...**) so a forty-minute chart does not hitch the catalog at 60 FPS.

What ships

Desktop is roughly an **8 MB** binary; the web package is about **1.8 MB zipped** (~4 MB wasm + JS + a ~420 KB preload of fonts/shaders/logo — songs never ride along). That size is mostly “link only what Rocksmith content needs” and keep assets out of the download, not a separate size project.

Stems & Demucs

Stem mode lets you mix six isolated tracks — drums, bass, other, vocals, guitar, piano — each with its own fader (0–200%). Mute the guitar and play over the rest of the band, or solo a part to learn by ear. Combined with pitch-preserving speed and section loops, this is the usual "drill this four bars" workflow.

Separation does **not** run inside Rock N Roller. It is done offline with the companion Demucs MLX Audio Stemmer, then the game mixes what it finds.

Demucs MLX on itch.io — nicapotato.itch.io/demucs-mlx-app

1. Companion app
2. Stemming workflow
3. How they link to PSARCs
4. Mixer in Rock N Roller
5. Find songs that have stems
6. Try it on the web

Companion app

Same raylib GUI on both platforms; the neural-net worker is what changes. Default model is Demucs `htdemucs_6s` (the six stems above). Input can be MP3, WAV, OGG, FLAC, M4A, or a Rocksmith `*_p.psarc` / `*_m.psarc` (it extracts the chart's audio first; Mac audio lives under `audio/mac`).

	MACOS APPLE SILICON	WINDOWS X64
Backend	demucs-mlx (MLX + Metal)	Official Meta Demucs + PyTorch CUDA
Download	~132 MB	~2 GB (ships the CUDA runtime)
Speed	Far faster than realtime (about 73× on an M4 Max for 4-stem `htdemucs`). Unified memory; no extra GPU.	NVIDIA GPU recommended (~5–7 GB VRAM for 6-stem). CPU works but is slow (a 4-minute track can take about a minute). AMD/Intel GPUs fall back to CPU.
Needs	Apple Silicon Mac. Unsigned: `xattr -cr "DemucsMLX.app"`	Recent NVIDIA **driver** (not a CUDA toolkit). SmartScreen: More info → Run anyway.

The Windows zip is large because MLX does not run on Windows, so the worker bundles PyTorch's CUDA kernels. You do not install a CUDA toolkit. 3–4 GB cards may run out of VRAM on the default 6-stem segments.

Stemming workflow

1. In Demucs MLX, add an MP3/WAV *or* a Rocksmith `*_p.psarc` or `*_m.psarc`.
2. Pick an **output folder**. Point it at `{library}/stems` on the same library folder Rock N Roller uses (or copy the six files there afterwards).
3. Run separation. For `Eagles_Hotel-California_v1_1_p.psarc` it writes:

```
{library}/stems/Eagles_Hotel-California_v1_1_p/
  drums.mp3  bass.mp3  other.mp3
  vocals.mp3  guitar.mp3  piano.mp3
```

4. In Rock N Roller Settings, bind that library folder (if you have not already) and run **Batch sync**. Folder layout: changelog. Settings: settings.

How they link to PSARCs

Rock N Roller takes the PSARC filename without `.psarc`, looks for `{library}/stems/{that_name}/`, and requires **all six** MP3s. Only then is the song marked `has_stems` in SQLite. Paths are indexed per song; your PSARC and stem audio stay on disk — the database only stores metadata.

Folder name must match the PSARC basename exactly. If the chart is `Foo_p.psarc`, the six files have to live in `Foo_p/`, not `Foo/` or a renamed folder. A Mac archive `Foo_m.psarc` writes `Foo_m/` the same way, so a PC copy and a Mac copy of one song keep separate stem folders.

Mixer in Rock N Roller

When a song has all six stem MP3s indexed, the **Stems** popup turns stem mode on and sets per-stem levels. Each fader runs **0–2.0** (shown as 0–200%; default 1.0) so you can mute or boost parts for practice. Playback opens six streaming MP3 decoders under one mixer that shares the same time-stretch path as normal audio, so practice speed still holds pitch. Long songs stream stems with seek points instead of loading them whole.

Mixer controls: settings. Transport and loop keys: keybinds.

Find songs that have stems

Catalog search filters on the SQLite flag, not by reopening files:

- `stems="true"` — songs with all six external stem MP3s indexed
- `stems="true" tuning="E Standard | Drop D"` — stemmed charts in either tuning

Full query language: features.

Try it on the web

`?demo=stems` loads curated demo PSARCs plus 6-stem MP3 trees into `/weblib/DemoStems`. That root is **additive** — it does not replace any stems folders you already granted. It is a larger download than `?demo=songs` (charts only). Chromium required; nothing is uploaded.

How the rest of the engine fits together: features. Desktop builds: versions.

Keybinds

Keyboard shortcuts for the song library and — most usefully — for practice inside a song. Mouse handles the rest (seek bar, volume, layout, stems, cell settings).

In song (cheat sheet): `Space` / `K` pause · `J` / `L` ±10s · `,` restart / previous · `.` next song · `←` / `→` speed presets · `P` section loop · `Enter` / `0` jump to loop start · `[` `]` speed · `Z` zen mode · `Esc` back

Inside a song

These apply while a chart is open. Most transport keys are ignored while you're dragging a seek, loop, volume, or speed slider (or a cell-settings slider). Pause always works.

KEY	ACTION
`Space or K`	Pause / resume
`J`	Seek backward 10 seconds
`L`	Seek forward 10 seconds
`,`	Go to the start of the song (or the section-loop start, if loop is on). If you are already within ~2 s of that start, or you press `,` twice quickly, skip to the previous song in the queue.
`.`	Play the next song in the queue (same as the footer **>>** button). No-op for a one-shot PLAY FILE / loose track.
`← / →`	Step playback speed along presets 0.50 → 0.80 → 1.00 → 1.25 → 1.50 → 2.00 (clamp at ends; from a non-preset speed, snap to the next strictly lower/higher step)
`P`	Toggle **section loop** around the current song section (only if the chart has sections). Turning it on seeks to the loop start.
`Enter, keypad Enter, or 0`	Jump to the loop start and unpause (only when section loop is on)
`[`	Playback speed −0.1× (clamped 0.3–3.0)
`]`	Playback speed +0.1× (clamped 0.3–3.0)

KEY	ACTION
`Z`	Toggle **zen mode**: hide the header and footer so the 3D highway fills the whole window (only the ZEN and Catalog buttons stay). Same as the ZEN header button.
`Esc`	Close an open play popup → else cancel speed typing → else return to the catalog

Typing a speed value

Click the speed number in the footer to type a rate directly (0.3–3.0). While that field is focused:

KEY	ACTION
`Digits / .`	Edit the value (one decimal point)
`Backspace`	Delete last character
`Enter`	Commit if in range; otherwise discard focus
`Esc`	Revert to the current speed and unfocus

Footer sliders (Vol, Speed, seek bar, loop edges) and the Catalog / Loop / Auto-cycle buttons are mouse-only, as are cell header menus: Layout, arrangement source, viz mode, Stems, Cell Settings. Previous / next also have `,` / `.`; the footer restart button is still mouse-only (keyboard restart is a single `,`).

Catalog (song library)

Song table

Library nav, browse mode, search field not focused:

KEY	ACTION
`↑ / ↓`	Move selection (hold to repeat; table auto-scrolls)
`Enter`	Load / play the hovered song
`Esc`	Dismiss nested UI in order: playlist edit → playlist popout → profile popout → else leave Albums / Artists / Settings back to Library

Playlist panel open

KEY	ACTION
`↑ / ↓`	Scroll the playlist list
`Page Up / Page Down`	Scroll by one viewport

Search field focused

KEY	ACTION
`Printable keys`	Insert at caret (or replace selection)
`Backspace / Delete`	Delete selection or character before/after caret
`← / →`	Move caret; Shift extends selection
`Home / End`	Jump to start/end; Shift extends selection
`Ctrl/Cmd+A / C / X / V`	Select all / copy / cut / paste (ASCII printable)
`Esc`	Clear search and unfocus

Albums / Artists filter

When that card's search is focused: type to filter, `Backspace` to delete, `Esc` to clear and unfocus.

Name / confirm modals

KEY	ACTION
`Printable keys`	Type into the name field
`Backspace`	Delete last character
`Enter`	OK (empty name is rejected)
`Esc`	Cancel

Esc never quits the app — only the window close control does. See also settings and features.

Settings

What the different options mean — library folders and stems in the catalog Settings screen, autoplay, and the per-cell highway / TabSheet / lyrics controls while a song is open.

1. Library & stems
2. Autoplay
3. Profiles & volume
4. While playing
5. Cell Settings
6. Layout, source & viz
7. Stem mixer

Library & stems

Open **Settings** in the catalog nav. Point the app at **one library folder**: a Rocksmith 2014 install if you own the game, or any folder with `dlc/` and `stems/`. Steam defaults: macOS `~/Library/Application Support/Steam/steamapps/common/Rocksmith2014`, Windows `C:\Program Files (x86)\Steam\steamapps\common\Rocksmith2014`, Linux `~/.steam/steam/steamapps/common/Rocksmith2014`. Settings in the app shows the expected path for your OS. Folder layout: 0.2.0 changelog.

Web player: Chrome will not open `Program Files`, `AppData` or `~/Library`, so the Steam paths above cannot be chosen in the browser. Copy `songs.psarc` + `dlc/` (+ `stems/`) to a folder such as `Documents/RockNRoller` and choose that. Details: Folders Chrome cannot open.

CONTROL	MEANING
Library folder	One root. Charts are scanned recursively from `dlc/` (`dlc/cdlc/…` is fine). Stem mixes live in `stems/{psarc_basename}/` with six MP3s. Clear drops the bound folder (files on disk stay).
Choose / Change library folder	Pick that root. If you own Rocksmith 2014, choose the Steam `Rocksmith2014` folder itself, not its `dlc/` subfolder (on desktop the picker opens there when it exists, else in `steamapps/common`). Otherwise choose any folder — `dlc/` and `stems/` are created if missing. PC `*_p.psarc` and Mac `*_m.psarc` both play on any computer. On the web this uses Chromium's folder picker (read-only), which refuses system and app-data folders — use a copy outside them.
Open folder	Reveal the bound library in Finder or Explorer so you can drop charts into `dlc/`. Desktop only.
Restore library (web)	Re-request access to a folder you granted before (browser permission).

CONTROL	MEANING
Batch sync / Resync library	Rescan `dlc/` (recursive) and `stems/`. Unchanged files are skipped via the size/mtime cache; new or modified charts are re-parsed.
Reset	Wipe profiles, playlists, and the library cache after confirm. Destructive — your PSARC files on disk are not deleted.
Set all to Default / Inverted	Apply string orientation for all cell size classes: default = red/low-E at **top**; inverted = red at **bottom**. Same invert flag as Cell Settings.
Set all to Right-handed / Left-handed	Apply handedness for all cell size classes. Left-handed mirrors fret positions about the neck midpoint (Rocksmith-style lefty). Same `left_hand` flag as Cell Settings.

Autoplay

Cycled from the catalog Settings **Auto: …** button or the play footer **Auto** control. Controls what happens when a track ends.

MODE	BEHAVIOR
Auto: off	Stop at the end of the track
Auto: loop	Restart the current song
Auto: next	Play the next song in the filtered queue (default)
Auto: shuf	Pick a random next song
Auto: shuf*	Shuffle without immediately repeating the same song

Profiles & volume

The **Profile** button opens a popout to switch, create, or delete profiles. Each profile stores its own playlists, cell settings, and master volume.

SETTING	MEANING
Game volume (Vol)	Master playback level, 0–1 (default 1). Adjusted with the play footer Vol slider; saved to the active profile when you release the slider.

While playing

Footer and cell-header controls that aren't on the catalog Settings page:

CONTROL	MEANING
Speed	Practice tempo without changing pitch, **0.3×–3.0×** (default 1.0). Slider, typed field, footer **1x** reset, or `[` / `]` keys. See keybinds.
Loop	Section loop on/off — same as `P`. Needs chart sections; loops the section under the playhead. Drag loop edges on the seek bar when enabled. `Enter` / `0` jumps to loop start. See features § loop.
Transport	Previous / play-pause / restart / next, plus Catalog to leave the song.
Zen	Distraction-free mode — same as `Z`. Hides the header and footer so the 3D highway fills the whole window; only the floating **ZEN** and **CATALOG** buttons stay. Toggle the button (top right of the header, next to Catalog) or press `Z` again to bring the controls back. Great for portrait / mobile screens.

Cell Settings

Opened from a cell's **CELL SETTINGS** header. Values apply to that cell's **size class** (Quarter / Tall / Wide / Full) for the current profile — not just the one song.

CONTROL	MEANING	RANGE	DEFAULT
Zoom / Tab zoom	While the cell is on the 3D highway: camera distance / highway scale. While it is on **TAB**: TabSheet page scale. The two values are stored separately (`zoom` vs `tab_zoom`) and do not copy across when you flip the toggle.	0.5–2.5	1.0
Runway angle	How steep the highway grade feels (camera elevation)	0.35–2.2	1.0
Render distance	How many seconds of notes are drawn ahead of the strike line	1.5–6.0 s	3.0 s
Bar lines (measure starts)	Horizontal cyan strips on the 3D highway at each SNG downbeat — one line per bar, not per beat. Same beat map as TabSheet measures. See features § bar lines.	off / on	on
Lyric size	Font scale for lyric strips / lyric cells	0.6–2.0	1.0
Lyric lines	How many lyric rows are visible	1–8	3

CONTROL	MEANING	RANGE	DEFAULT
Inverted (red at bottom)	String order. Off: red / low-E at top (Rocksmith-style default). On: red at bottom.	off / on	off
Left-handed	Mirror frets about the neck midpoint for lefty players. Off = right-handed (default).	off / on	off
Reset (per row)	Restore that field's default	—	—

Quarter instrument cells expose zoom / angle / render / invert / left-handed. Lyrics-only cells expose lyric size and lines. Larger instrument cells expose the full set. Also see features § lefty.

Layout, source & viz

CONTROL	MEANING
Layout	Chart grid template, e.g. Full; vertical/horizontal split; 4 Quarter; 3 highways + lyrics; tall/wide + quarter mixes. Optional **Auto-fill cells** assigns arrangements into empty slots. Saved with chart layout meta.
Cell source	What that pane shows: Empty, Lead, Rhythm, Bass, Combo (lead + rhythm on one chart, listed only when the song has it), bonus parts (`+L` , `+R` , …), or Lyrics. See features § arrangements.
Cell viz	How the pane is drawn. Instrument cells flip with the header **TAB** toggle: **3D** highway or wrapping **TabSheet** (rhythm notation inferred from note times + the SNG beat grid; see features § TabSheet). **3D + Lyrics** is the inline lyric strip on the highway (no chord names). Lyrics cells: **Lyrics** or **Chords + Lyrics** (named chords above the words). See features § Chords + Lyrics.

Stem mixer

When a song has all six stem MP3s indexed, the **Stems** popup can turn stem mode on and set per-stem levels (drums, bass, other, vocals, guitar, piano). Each fader runs **0–2.0** (default 1.0) so you can boost or mute parts for practice. Stem separation itself is offline (Demucs); the app only mixes what it finds. See stems & Demucs.

Keyboard shortcuts: keybinds. How the pieces fit together: features.